EDUFLOW– STUDENT PERFORMANCE AND ATTENDANCE ANALYTICS SYSTEM

Name : *HADIAH TASNEEM*

Project : *EduFlow – Student Performance and Attendance Analytics System*

Course : *Full-Stack JAVA*

ABSTRACT

EduFlow is a comprehensive full-stack web application developed to automate and streamline student academic management processes. The system integrates modern technologies such as **Spring Boot, React.js, and MySQL** to provide a scalable and efficient solution.

The application enables:

- Secure user authentication using OTP verification
- Role-based access for Admin, Teacher, and Student
- Efficient management of student records
- Real-time attendance tracking
- Marks entry and performance calculation
- Advanced analytics for student performance evaluation
- Automated PDF report generation

This system reduces manual errors, improves data accessibility, and enhances decision-making through analytics.

1. INTRODUCTION

Traditional academic systems rely heavily on manual processes, which are time-consuming, inefficient, and prone to errors. EduFlow addresses these issues by providing a centralized digital platform.

The system ensures:

- Data consistency
- Faster processing
- Improved transparency

1.1 Objectives

- To design and develop a full-stack academic management system
- To automate attendance and marks recording
- To provide real-time analytics and reports
- To enhance data security using authentication mechanisms

1.2 Scope

The system can be deployed in:

- Schools
- Colleges
- Training institutes

It supports multi-user access and can handle large datasets efficiently.

2. SYSTEM ARCHITECTURE

EduTrack follows a **three-tier layered architecture**, ensuring modularity and scalability.

Layers Explanation

1. Presentation Layer (Frontend)

- Built using React.js
- Handles user interaction
- Displays dynamic content

2. Business Logic Layer (Backend)

- Developed using Spring Boot
- Processes user requests
- Implements business rules

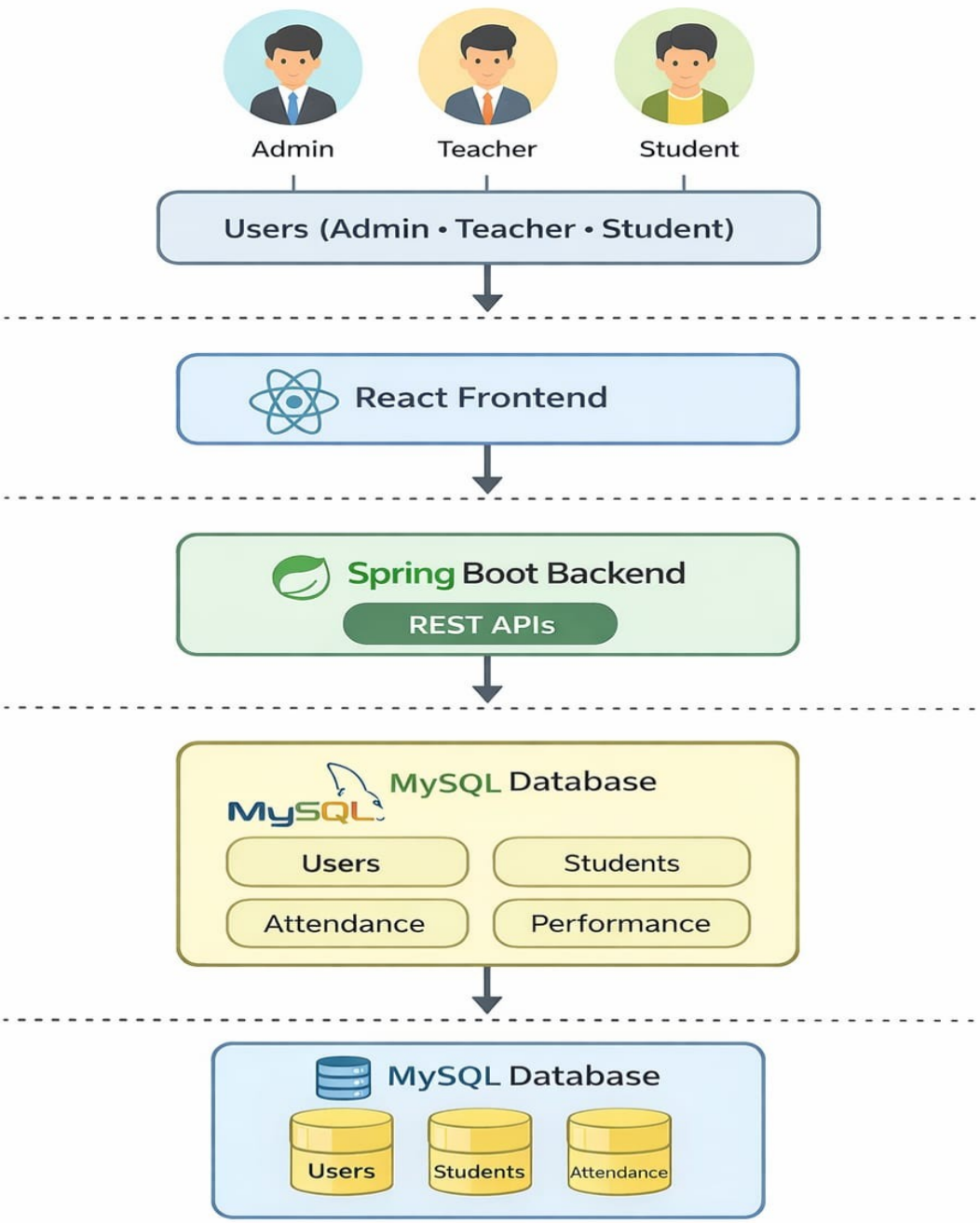
3. Data Layer (Database)

- MySQL database
- Stores structured data

4. Architecture Diagram

This diagram represents the three-tier architecture of the system, consisting of the frontend (React), backend (Spring Boot), and database (MySQL). It shows how data flows between layers and how the system processes user requests.

Figure 1: System Architecture of EduFlow



3. TECHNOLOGIES USED

Technology	Description
React.js	Used for building interactive UI
Spring Boot	Backend framework for REST APIs
MySQL	Relational database
JPA/Hibernate	ORM for database operations
BCrypt	Password encryption
OpenPDF	PDF report generation

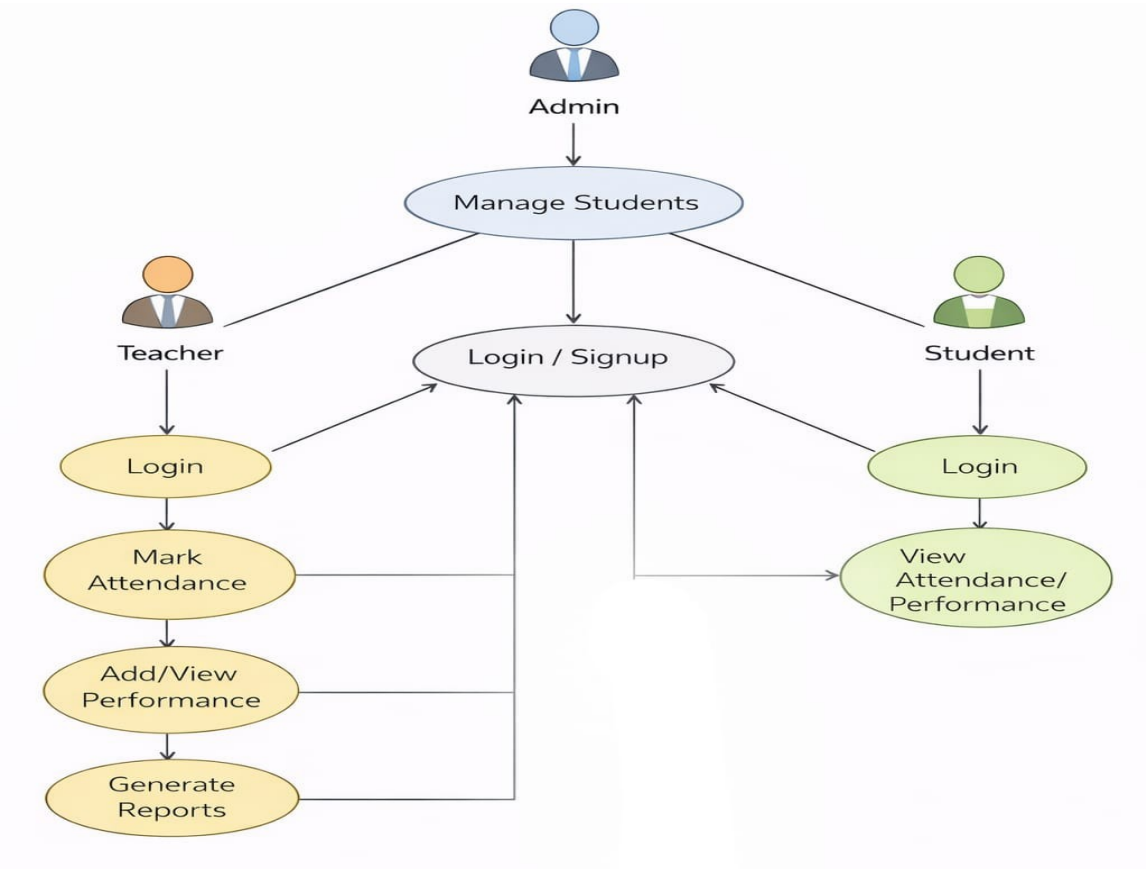
4. SYSTEM DESIGN

4.1 Use Case Diagram

This diagram represents the interaction between different actors (Admin, Teacher, and Student) and the system functionalities. It illustrates how users perform operations such as login, managing students, marking attendance, entering marks, and viewing analytics. The diagram helps in understanding the overall system behavior and user interactions.Actors:

- Admin → manages system
- Teacher → manages students
- Student → views data

Figure 2: Use Case Diagram of EduFlow System



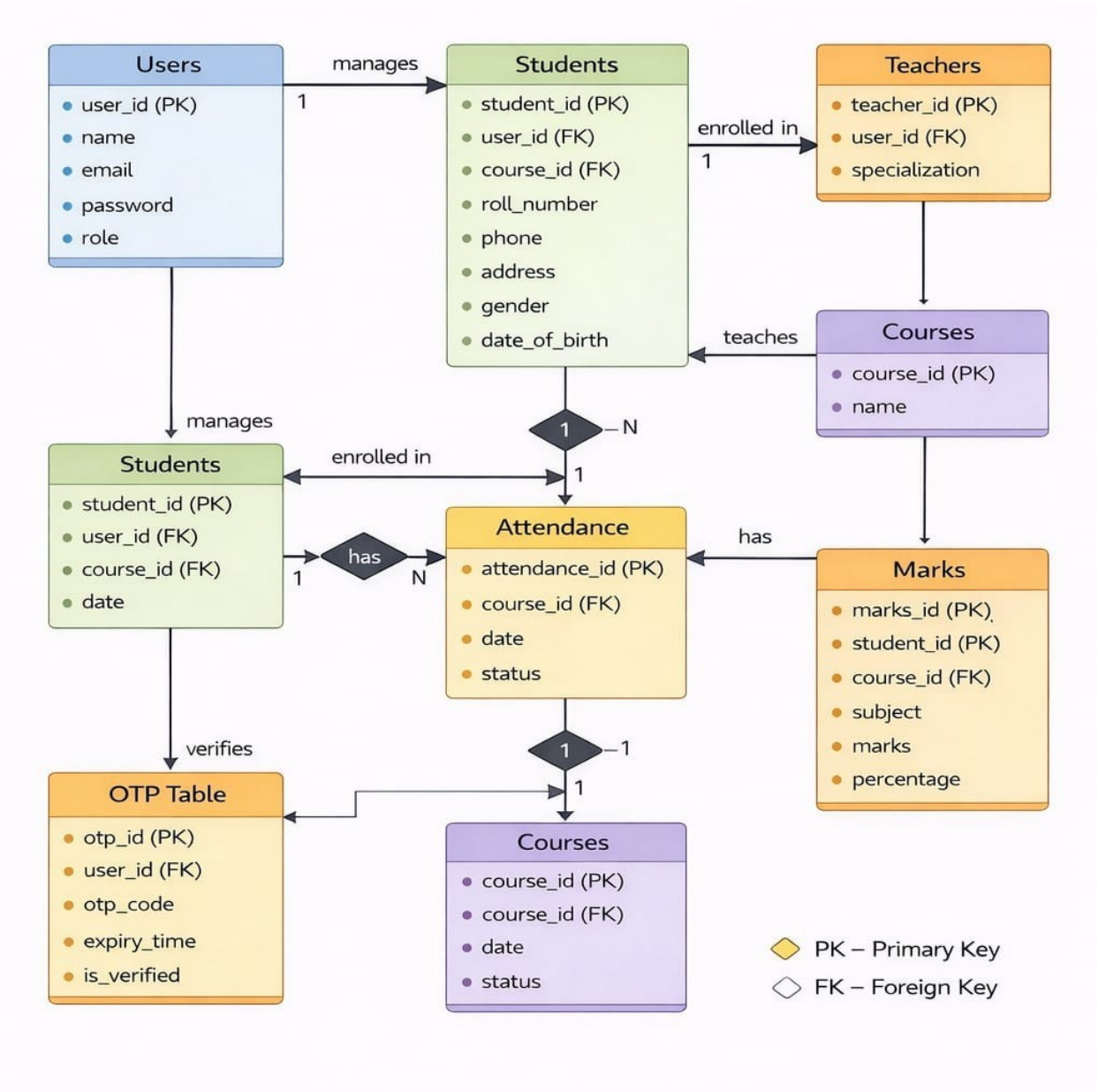
4.2 ER Diagram

This diagram shows the structure of the database and the relationships between different entities such as User, Student, Teacher, Attendance, and Marks. It highlights primary keys, foreign keys, and the associations between tables, ensuring proper data organization and integrity within the system.

Shows relationships between:

- User ↔ Student
- Student ↔ Attendance
- Student ↔ Marks

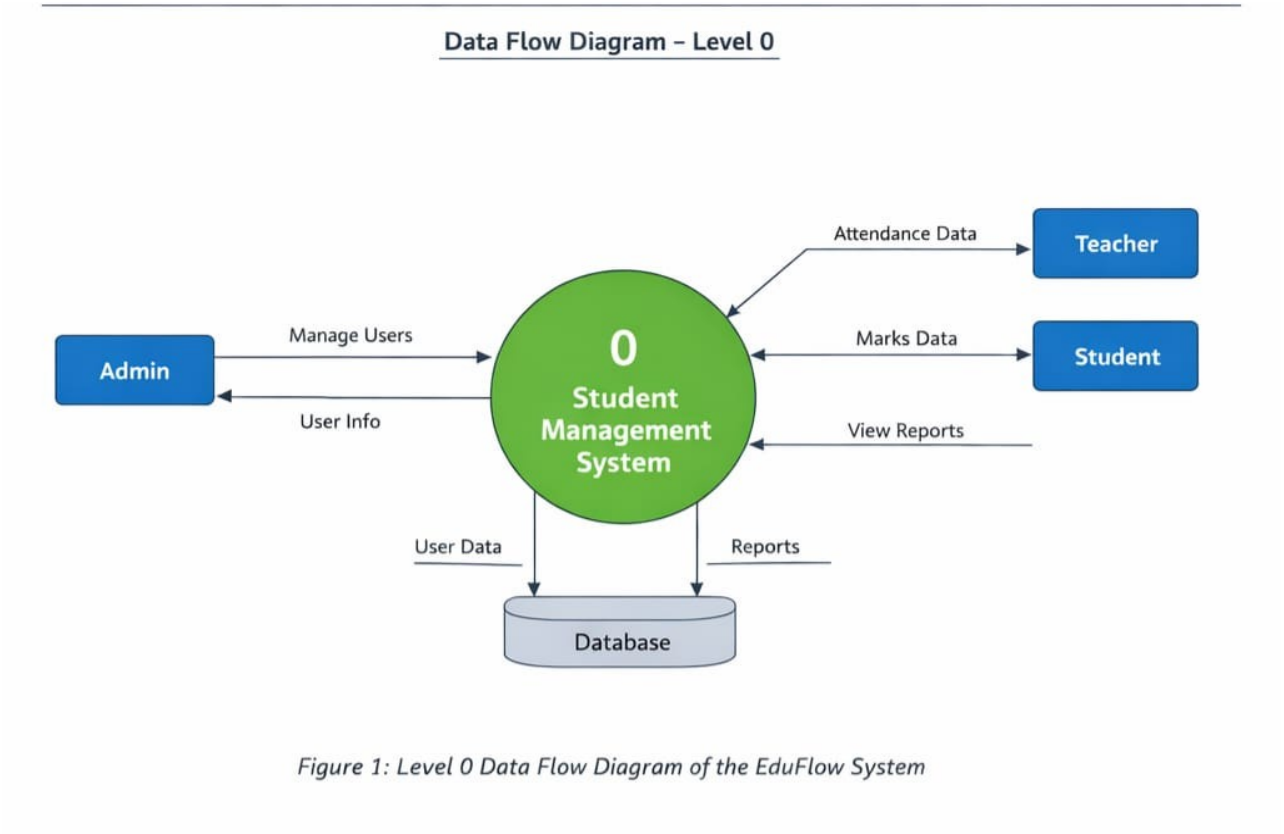
Figure 3: Entity Relationship (ER) Diagram of EduFlow Database



4.3 Data Flow Diagram

This diagram illustrates how data moves between users, processes, and the database within the EduFlow system.

Figure 4: Data Flow Diagram (DFD) of EduFlow System



5. MODULE DESCRIPTION

5.1 Authentication Module

This module ensures secure system access.

Features:

- OTP-based email verification
- Password encryption using BCrypt
- Login validation
- Forgot password functionality

Importance:

- Prevents unauthorized access

- Ensures data privacy

5.2 Student Management Module

Functionalities:

- Add new student
- Update student details
- Delete student records
- Search students

Benefits:

- Centralized student database
- Easy data retrieval

5.3 Attendance Module

Functionalities:

- Mark attendance daily
- Store attendance records
- Track attendance history

Logic:

Attendance percentage =
(Present Days / Total Days) × 100

5.4 Marks Module

Functionalities:

- Add subject-wise marks
- Calculate percentage
- Store exam details

Benefits:

- Easy performance tracking
- Structured data storage

5.5 Analytics Module

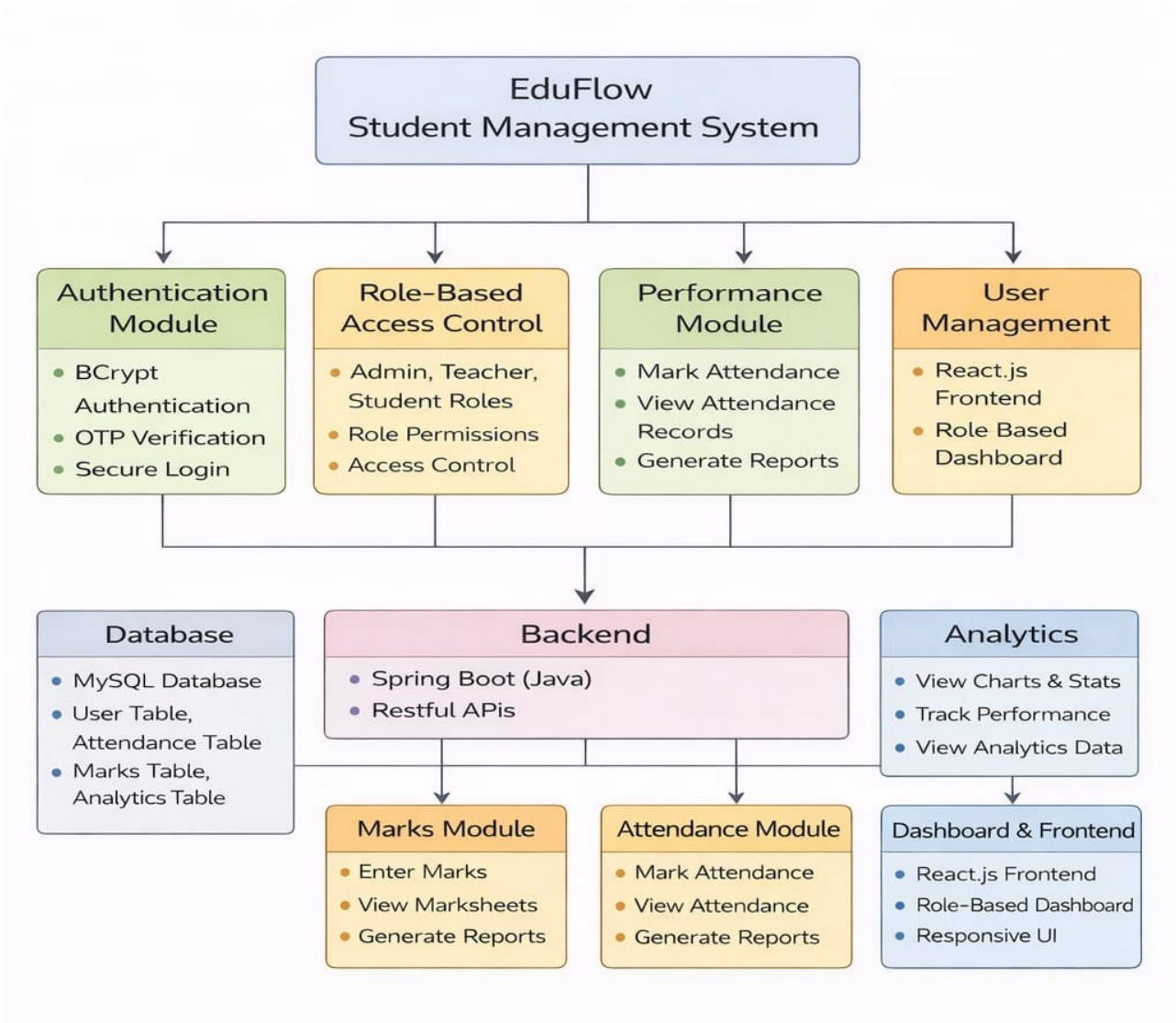
Functionalities:

- Calculate average marks
- Calculate attendance percentage
- Classify performance

Performance Levels:

- Good → ≥ 75%
- Average → 50–74%
- Poor → < 50%

Figure 5: Module Diagram of EduFlow System



6. DETAILED PROJECT STRUCTURE

6.1 BACKEND STRUCTURE (SPRING BOOT)

The backend of EduFlow is developed using **Spring Boot**, which follows a layered architecture to ensure modularity, scalability, and maintainability. Each layer has a specific responsibility, making the system easy to manage and extend.

```
com.eduflow
├── controller/
├── service/
├── repository/
├── model/
├── dto/
├── config/
└── exception/
```

This structure ensures **separation of concerns**, where each component handles a specific task independently.

6.1.1 Controller Layer

The Controller layer acts as the **entry point of the backend system**. It is responsible for handling incoming HTTP requests from the frontend and returning appropriate responses.

Key Functions:

- Receives client requests (GET, POST, PUT, DELETE)
- Maps URLs to specific methods using annotations like `@GetMapping`, `@PostMapping`
- Validates input data using annotations like `@Valid`
- Sends response data back to the frontend in JSON format

Example Responsibilities:

- Handling login requests
- Fetching student data
- Submitting attendance

Importance:

This layer ensures proper communication between frontend and backend and maintains a clean API structure.

6.1.2 Service Layer

The Service layer contains the **core business logic of the application**. It acts as an intermediate layer between the Controller and Repository.

Key Functions:

- Processes and manipulates data
- Implements business rules
- Performs calculations (e.g., attendance %, average marks)
- Ensures validation before saving data

Responsibilities:

- Data transformation between DTO and Entity
- Applying conditions and validations
- Coordinating multiple repository calls

Example:

- Calculating performance status (Good/Average/Poor)
- Processing OTP verification

Importance:

This layer ensures that business logic is centralized and reusable.

6.1.3 Repository Layer

The Repository layer is responsible for **interacting with the database**. It uses **Spring Data JPA** to simplify database operations.

Key Functions:

- Performs CRUD operations (Create, Read, Update, Delete)
- Executes database queries
- Retrieves and stores data

Responsibilities:

- Fetching student records
- Saving attendance data
- Querying marks and analytics

Advantages:

- Reduces boilerplate code
- Provides built-in methods like `findById()`, `save()`

6.1.4 Model Layer (Entities)

The Model layer represents the **database structure in the form of Java classes**.

Key Features:

- Uses annotations like `@Entity`, `@Table`, `@Id`
- Defines relationships such as:
  - One-to-One
  - One-to-Many
  - Many-to-One

Example:

- User entity
- Student entity

Importance:

Ensures proper mapping between application objects and database tables.

6.1.5 DTO Layer (Data Transfer Objects)

DTOs are used to **transfer data between frontend and backend securely**.

Key Functions:

- Prevents direct exposure of database entities
- Controls what data is sent to the client
- Improves security and flexibility

Example:

- Sending only required fields like name and email instead of full entity

6.1.6 Configuration Layer

This layer manages application-level configurations.

Includes:

- Security configuration (authentication rules)
- CORS configuration (frontend-backend communication)
- Application settings

Importance:

Ensures secure and smooth operation of the system.

6.1.7 Exception Handling

Centralized exception handling improves system reliability.

Key Functions:

- Handles runtime errors

- Provides meaningful error messages
- Prevents system crashes

Benefits:

- Better debugging
- Improved user experience

6.2 FRONTEND STRUCTURE (REACT)

The frontend of EduFlow is developed using **React.js**, which follows a component-based architecture.

```
src/
├── pages/
├── components/
├── api/
├── context/
├── App.jsx
└── main.jsx
```

This structure ensures **reusability, scalability, and clean UI development**.

6.2.1 Pages

Pages represent the **main screens of the application**.

Features:

- Each page is linked to a route
- Contains UI and logic for a specific feature

Examples:

- Login Page → User authentication
- Dashboard → Overview of system
- Students Page → Manage student data

6.2.2 Components

Components are **reusable UI elements** used across multiple pages.

Examples:

- Navbar → Top navigation
- Sidebar → Menu navigation
- Cards → Display statistics

Advantages:

- Reduces code duplication
- Improves maintainability

6.2.3 API Layer

The API layer handles communication between frontend and backend.

Key Features:

- Uses Axios for HTTP requests
- Sends and receives JSON data
- Handles API responses and errors

Importance:

Ensures smooth data exchange between client and server.

6.2.4 Context API

Used for **global state management**.

Stores:

- User login data
- Authentication status

Benefits:

- Avoids prop drilling
- Provides centralized state management

6.2.5 Routing

Routing is managed using **React Router**.

Functions:

- Enables navigation between pages
- Protects routes based on user roles

Example:

- Admin → Admin dashboard
- Student → Student dashboard

7. DATABASE DESIGN

The database is designed using **relational schema** to maintain data integrity and relationships.

7.1 USERS TABLE

This table stores authentication and role information.

Explanation:

- Each user has a unique email
- Role determines access (Admin/Teacher/Student)
- Password is stored in encrypted form

Importance:

Acts as the base table for authentication.

7.2 STUDENTS TABLE

Stores detailed student information.

Explanation:

- Linked to Users table via `user_id`
- Contains academic and personal details

Importance:

Separates login data from student data for better organization.

7.3 TEACHERS TABLE

Stores teacher-related information.

Explanation:

- Linked to Users table
- Contains subject and qualification

7.4 ATTENDANCE TABLE

Stores attendance records.

Explanation:

- Each record represents a student’s attendance for a specific date
- Helps calculate attendance percentage

7.5 MARKS TABLE

Stores academic performance data.

Explanation:

- Stores subject-wise marks
- Used for analytics calculations

7.6 COURSE TABLE

Stores course information.

Explanation:

- Defines course structure
- Linked with students

7.7 OTP TABLE

Used for authentication security.

Explanation:

- Stores OTP temporarily
- Includes expiry time for security

8. IMPLEMENTATION

8.1 Backend Implementation

- Developed using Spring Boot framework
- REST APIs implemented
- Uses dependency injection for loose coupling
- JPA used for database interaction

8.2 Frontend Implementation

- Built using React.js
- Component-based UI
- Responsive design for multiple devices
- API integration using Axios

9. TESTING

Types of Testing:

- **Unit Testing:** Tests individual components
- **Integration Testing:** Tests module interaction
- **System Testing:** Tests complete system functionality

10. RESULTS

The system successfully:

- Manages student data efficiently
- Tracks attendance accurately
- Generates reports automatically

- Provides performance analytics

11. FUTURE ENHANCEMENTS

- Mobile application development
- AI-based performance prediction
- Cloud deployment for scalability

12. CONCLUSION

EduFlow provides a robust and scalable solution for managing academic activities. It reduces manual effort, enhances accuracy, and improves overall system efficiency.